Avalux: An Open Integrated Platform for Moderated Usability Testing with Real-Time Evaluator–Participant Synchronization

Hugo Correia
Affiliation placeholder
City, Country
email@placeholder.pt

***Abstract*—Moderated usability testing remains the reference method for discovering interaction problems, yet its instrumentation is fragmented: evaluators juggle stopwatches, spreadsheets, questionnaire tools, and manual report assembly, while commercial platforms focus on unmoderated remote studies whose moderated modes capture video and free-form notes rather than structured per-task metrics. We present Avalux, an open, self-hostable web platform that integrates the complete moderated-testing pipeline: reusable test protocols (tasks with optimal-path baselines, typed error taxonomies, per-task questions including in-browser audio/video/photo capture, interview scripts, and study-specific participant attributes); a live evaluator cockpit that logs per-task completion time, action counts, categorized errors, hesitation events, and Single Ease Question ratings; and a participant client that runs on the participant's own device via a join link, synchronized with the evaluator in real time over WebSocket channels, including a gating protocol that prevents the session from advancing while the participant is still answering. Sessions conclude with an automatically scored System Usability Scale questionnaire and a semi-structured interview, and export to a self-contained PDF report. We describe the architecture, a React single-page application over a Postgres backend with row-level security, and report two case studies: a four-participant evaluation of a household touch-panel controller (mean SUS 28.1) and a two-participant evaluation of an eye-tracking analysis dashboard (SUS 80). The studies show that the platform carries complete, realistic evaluations end to end, including a physical device beyond the reach of browser-based instrumentation, while producing diagnostic task-level detail alongside the standard summary metrics.**

***Index Terms*—usability testing, moderated evaluation, SUS, SEQ, research tools, human–computer interaction**

I. Introduction

Task-based moderated testing with a small number of participants remains the workhorse of usability practice: a facilitator observes a participant attempting representative tasks, records performance and difficulties, and complements observation with standardized instruments such as the System Usability Scale (SUS) [1] and the Single Ease Question (SEQ) [2]. Classic problem-discovery models show that even four to five participants uncover the majority of severe problems [3], [4], which keeps the method attractive for small teams and academic projects.

The tooling situation is less attractive. Commercial platforms concentrate on unmoderated remote studies, where metrics are captured by automatic instrumentation of the participant's browser; their moderated modes produce session videos and free-form notes that must be coded after the fact. In practice, a moderated session is therefore run with an ad-hoc toolchain (a stopwatch, a paper or spreadsheet observation grid, a separate questionnaire form, and manual report writing) that is slow, error-prone, and hard to standardize across studies.

This paper presents *Avalux*, an open web platform that integrates the moderated-testing pipeline end to end. Its contributions are:

- an **integrated protocol model**: reusable templates couple tasks (with optimal time and action-count baselines) to typed error taxonomies, per-task questions (open, choice, rating, and in-browser audio/video/photo capture), interview scripts, and study-specific participant attributes;
- a **live moderated session** design in which the evaluator's cockpit (timer, action counter, one-tap typed error and hesitation logging, per-task SEQ) and the participant's own device are synchronized in real time, including a gating protocol that blocks task advancement while the participant is still answering the previous task's questions;
- **two case studies**, a household touch-panel controller and an eye-tracking analysis dashboard, both run entirely on the platform, showing that it carries realistic studies end to end, including a physical device beyond browser instrumentation, and that its task-level instruments localize the problems behind widely differing outcomes (mean SUS 28.1 vs. 80).

Avalux is deployed at https://avalux.pt; its schema, protocol seeds, and report generator are versioned with the application source.

II. Related Work

A mature market of commercial software-as-a-service platforms now supports remote usability evaluation. UserTesting combines a proprietary participant panel with video-first moderated ("Live Conversation") and unmoderated studies overlaid with AI-assisted analysis [5], and has absorbed the quantitative benchmarking platform UserZoom following their 2023 merger [6]. Maze is unmoderated by default, converting prototypes and live websites into task-based studies scored through automatically instrumented metrics such as completion rate, misclick rate, and task duration [7]. Lookback specializes

in live moderated sessions with screen, camera, and audio capture, stakeholder observation rooms, and timestamped free-form notes [8]; Loop11 supports both modes and, unusually, auto-computes SUS and NPS for unmoderated tests [9]; Useberry targets prototype-centric unmoderated testing with click and flow analytics [10]. Across these tools, quantitative evidence derives almost entirely from automated client-side instrumentation in unmoderated studies, while moderated modes yield video recordings and free-form notes that must be coded after the fact. To the best of our knowledge, none *combines* structured real-time evaluator logging in moderated sessions, per-task ease ratings integrated into the same pipeline, and an open, self-hostable architecture in one platform.

Academic work on remote evaluation dates to Hartson et al., who framed the network as an extension of the usability laboratory [11]. Andreasen et al. subsequently found remote synchronous testing "virtually equivalent" to conventional lab-based think-aloud, whereas asynchronous methods identify fewer problems at lower moderator cost [12], an asymmetry confirmed by Bruun et al. [13] and persisting in recent comparisons [14]. On the tooling side, RemUSINE and WebRemUSINE pioneered task-level instrumentation by comparing interaction logs against task models, but operate post hoc rather than steering live sessions [15], [16]. TechSmith's Morae became the de facto observation suite in usability practice [17], [18]; its Observer component supported real-time marker and error logging by observers during moderated sessions and is the closest precedent to our evaluator instrument, but Morae is a discontinued desktop product requiring installed software on the participant's machine, without participant-side questioning or an open architecture. More recent browser-based platforms such as UTAssistant [19], [20] and the analytics-augmented test-management system of Generosi et al. [21] target largely asynchronous workflows, and an action-research deployment of remote testing found the evaluation tool itself to be an obstacle to high-quality results [22].

Avalux's measurement model follows the ISO 9241-11 decomposition of usability into effectiveness, efficiency, and satisfaction [23]. Perceived usability is captured with the ten-item SUS [1], whose reliability [24], [25], benchmark mean of 68 [26], and adjective-based interpretation [27] are well established; per-task difficulty uses the Single Ease Question, shown to perform comparably to more elaborate post-task instruments [2]. Completion rate, time-on-task, and action counts relative to an optimal path are the standard objective performance metrics [28], [29], and moderated testing with small samples is supported by classic problem-discovery models [3], [4].

A final thread concerns where sessions take place and on whose device. Early comparisons found no significant remote-versus-local differences in critical errors or time-on-task [30], [31], while lab-versus-field studies of mobile applications show that removing participants from their natural device context can mask context-dependent problems [32], [33]. The COVID-19 pandemic pushed moderated protocols onto general-purpose videoconferencing [34], [35], with studies of remote moderation identifying the moderator's reduced visibility into the participant's device and context as its principal cost [36].

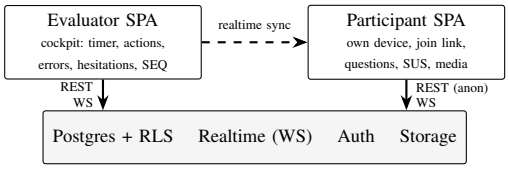

Fig. 1. Architecture: two browser roles over a shared Postgres backend; row-level security scopes evaluator data to accounts and participant access to per-session join codes; realtime channels synchronize both views.

Avalux addresses the gap these threads leave open. It is an open, integrated platform for moderated testing in which the evaluator live-logs, per task, completion time, action counts, typed errors, hesitation events, and SEQ ratings, while the participant completes tasks on their own device via a browser join link, with task state synchronized between the two roles in real time over WebSockets. A single pipeline carries a session from protocol definition through per-task SEQ, post-session SUS, and a semi-structured interview to an automatically generated PDF report. Unlike the commercial services surveyed above, Avalux is open and self-hostable (a React single-page application over Supabase Postgres with row-level security and realtime channels), and unlike prior academic tooling it makes fine-grained evaluator-side logging and evaluator–participant realtime synchronization first-class features of the moderated session rather than post hoc analyses.

III. System Design

A. Architecture

Avalux is a TypeScript React single-page application (React 19 with TanStack Router and Query, Tailwind CSS, shadcn/ui components, and Recharts visualizations, built with Vite) served statically from a CDN (Vercel), with all persistent state in a hosted Postgres instance (Supabase) accessed through PostgREST and secured by row-level security (RLS) policies (Fig. 1). Reports are rendered client-side with jsPDF, and the codebase is exercised by a unit test suite (Vitest) run on every push through continuous integration (GitHub Actions). Three backend primitives carry the design: (i) *RLS dual-role security*: evaluator rows are scoped to their authenticated account, while participants operate anonymously, authorized per session through unguessable join codes checked inside the policies themselves; (ii) *realtime channels*: the participant client subscribes to Postgres change streams over WebSockets and follows the evaluator's mutations (task advanced, status changed) without polling, while the evaluator's cockpit refreshes the participant-progress gate by short-interval polling of the same persisted state; (iii) *storage buckets* hold media answers captured in the participant's browser via `getUserMedia`. There is no custom application server, which keeps the platform self-hostable on free tiers of commodity services.

B. Protocol Model

A *template* is a reusable test protocol: an ordered set of *tasks*, optionally grouped, each carrying instructions and optimal time/action baselines; a study-specific *error taxonomy* (code plus label, e.g. *WB, wrong button*); *per-task questions* of seven types (open text, single and multiple choice, rating, audio, video, photo); a semi-structured *interview script*; and *participant fields*: extra attributes such as handedness or vision correction that the study collects per participant. Templates also select the post-session instruments a study administers: the SUS is always on, and studies can add the unweighted (raw) NASA-TLX workload scale [37] and the eight-item UEQ-S user experience questionnaire [38]. A *session* instantiates a template for one participant and accumulates per-task results (status, time, action count, error and hesitation logs with timestamps, SEQ), question answers, interview answers, and the item scores of every administered instrument. Participants can be enrolled directly, through single-use personal links, or through shared links that spawn one session per respondent, with configurable collection (or omission, for anonymous participation) of identity fields. Task presentation order is a per-session strategy chosen at creation: the template's fixed order, an independent random shuffle, or Latin-square rotation in which consecutive sessions rotate the starting task so that every task occupies every position equally often across the cohort; the strategy is recorded on the session and stated in its report.

C. Live Session and Synchronization

During a session the evaluator's cockpit drives the protocol (Fig. 3). Completing a task opens the SEQ prompt and advances a persisted task pointer; the participant's device follows that pointer and shows the active task's instructions, so the facilitator never has to read them aloud. When the evaluator marks a task complete, its questions unlock on the participant's device; conversely, the evaluator's success/partial/failure controls are *disabled* while the participant is still answering the previous task's questions, with a notification when they finish: a two-way gate designed to keep both roles aligned without requiring verbal coordination (Fig. 2). Both directions of the gate are computed from persisted state (the task pointer and the ratio of answers to questions per completed task), not from ephemeral messages, so either client can reload or lose connectivity mid-session and rejoin in a consistent state. The evaluator can step back one task without destroying recorded data, or explicitly reset a task (clearing its metrics, logs, and answers) after confirmation. After the last task the participant completes the interview and the SUS questionnaire on their own device; scores are computed and banded automatically [27], [26].

D. Measures

For each task i the platform records completion status $s_i \in \{\text{success}, \text{partial}, \text{failure}, \text{skipped}\}$, time on task t_i, evaluator-counted actions a_i, categorized error and hesitation events with within-task timestamps, and the SEQ rating

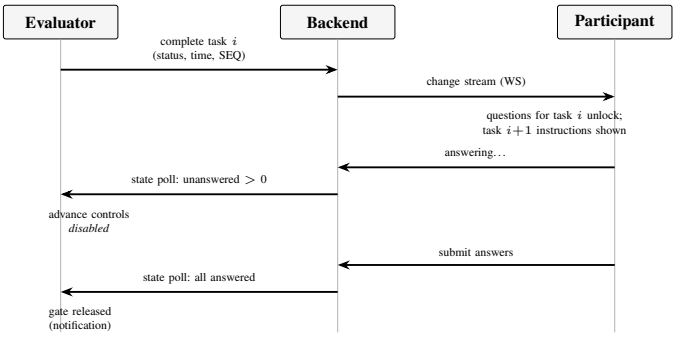

Fig. 2. Two-way gating during a moderated session. The participant follows the evaluator over a realtime channel; the evaluator's gate is refreshed by short-interval polling of the same persisted state, so either client can reload mid-session and rejoin consistently.

$q_i \in [1, 7]$. Templates supply per-task baselines t_i^* (optimal time) and a_i^* (optimal action count), following the optimal-path-referenced efficiency measures recommended in the measurement literature [28], [29]. Session-level indicators are the completion-status distribution, mean time and action efficiency $\frac{1}{|A|}\sum_{i \in A} t_i^*/t_i$ (and analogously for actions), where A contains only *attempted* tasks with $t_i > 0$ (skipped tasks are excluded rather than scored as zero-time successes, an aggregation rule we adopted after observing that including them produces unbounded efficiency values), and the SUS score with its adjective band [1], [27]. Cross-session SUS summaries are accompanied by two-sided 95% confidence intervals computed with the t-distribution, following the small-sample recommendation of Sauro and Lewis [29]. When a template administers them, raw NASA-TLX is scored as the unweighted mean of its six subscales [37] and UEQ-S as pragmatic, hedonic, and overall means on the -3 to $+3$ scale [38]. Distributions, efficiency ratios, and SUS scores are computed at display time from the stored task results; per-task error, hesitation, and action counts are persisted when the task completes, alongside the individual timestamped log entries they summarize.

Because evaluator-counted actions are a manual measure, the platform can optionally instrument browser-based systems under test directly: a dependency-free script embedded in the tested system streams click, keypress, and navigation events (element descriptors and key classes only, never typed text) into the running session, keyed by its join code and accepted server-side only while the session is in progress. The evaluator cockpit shows the automatic counts beside the manual tally during the session, the results view charts the captured event stream over session time, and the events export as a raw table, enabling a manual-versus-automatic count comparison as an internal validity check on the action measure.

E. Security Model

The platform's least conventional engineering choice is placing the whole authorization model inside the database. Evaluator tables carry policies of the form `user_id = auth.uid()`; participant access is granted to the anonymous

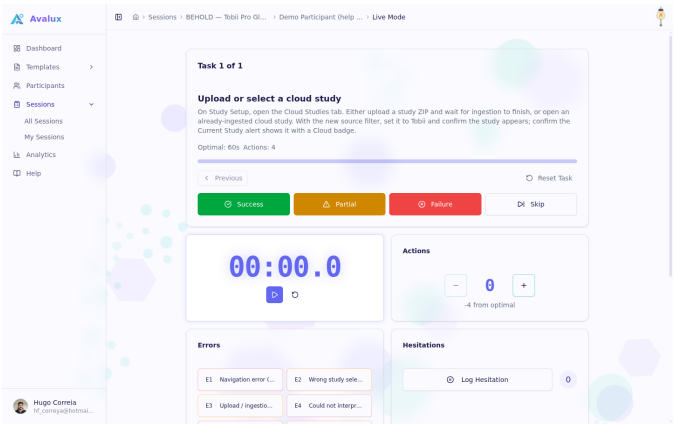

Fig. 3. Evaluator cockpit during a live session: task navigator with completion controls, timer, action counter, one-tap typed error logging, and hesitation logging.

role through policies that join the target row to a session whose unguessable `join_code` is non-null, so possession of a link is the capability to act in exactly one session. Because participants must also *write* (answers, SUS items, media references), each such table pairs a scoped `INSERT`/`UPDATE` policy with the read policy, and mutable invitation counters are further restricted with column-level grants so an anonymous client can increment a response counter but not rewrite an invitation's task selection. This model survived two studies without an application server, though it requires discipline: one gap, a missing anonymous update policy on invitation counters, which silently aborted link joins after the participant row was already created, was found during operation and is now covered by a regression migration. Participant media (photo, audio, and video answers) lives in a private bucket and is served exclusively through short-lived signed URLs, so a leaked link expires instead of exposing a recording permanently; deleting a session also deletes its stored media. Studies that must avoid personal data can disable identity collection entirely, producing anonymous participant records that never join the evaluator's roster, and a completed session's participant can be anonymized retroactively (identity replaced, contact details and custom attributes removed, demographics and metrics kept). Operational error monitoring follows the same stance: only uncaught exceptions are reported, tagged with opaque session identifiers, with no session replays, no performance traces, and no request payloads.

F. Analysis and Reporting

Session dashboards chart completion status, time and action counts against the template's optimal baselines, error and hesitation distributions, SEQ per task, and a normalized per-task performance radar (Fig. 4); per-task event timelines plot each logged error and hesitation at its within-task timestamp, showing *when* inside a task problems clustered; template-level analytics aggregate all completed sessions. A one-click export renders a self-contained PDF report (participant demographics, per-task tables, charts, timelines, logs, interview answers, and questionnaire breakdowns with cross-session comparison), replacing the manual assembly step that typically follows moderated studies. For external statistical analysis, every measurement a study produced can also be exported as raw flat tables (CSV or JSON): sessions, task results with baselines, timestamped event logs, question answers, and per-item questionnaire scores.

G. Qualitative Coding

Beyond the quantitative record, the platform supports a lightweight qualitative analysis workflow directly on collected text. Each template carries a code book: a set of short, color-coded labels with optional definitions, authored by the evaluator. Any open-ended task answer or post-session interview answer can then be tagged with any number of codes from the session's results view, and a code-frequency matrix (codes as rows, sessions as columns) aggregates tag counts at the template level as coding proceeds. Tags are exported as their own raw table alongside the quantitative data, with each tag row resolving the coded answer's text, source question, and session, so downstream analysis can move between counts and quotes. Commercial platforms typically confine tagging to recorded video timelines in their unmoderated products; coupling a template-scoped code book to the structured answers of moderated sessions keeps the coding unit aligned with the protocol itself.

H. Design Notes

Two design decisions proved more consequential than expected. First, *logging must not compete with observation*: every evaluator action during a task (counting an action, categorizing an error, marking a hesitation) is a single tap on a persistently visible control, sized to mobile touch-target guidelines so the cockpit is operable on a tablet held in the lap while watching the participant. Instruments that require navigation or typing (task questions, notes) are deferred to task boundaries. Second, *the protocol is an artifact, not configuration*: because a template fully encodes tasks, baselines, taxonomies, and instruments, it can be reviewed before the study like a questionnaire, reused across sessions unchanged, exported as printable observation sheets when a study must run entirely on paper, and, as in study 2, seeded from a written protocol document under version control.

IV. Case Studies

Two studies were run end to end on the platform. Both used in-person, co-located moderated sessions in which the participant handled the system under test while the evaluator logged performance in Avalux. Study 1 used the evaluator client alone; study 2 additionally used the participant's own device for questions, interview, and SUS via join links, so the synchronization and gating mechanism of Section III-C was exercised only in study 2's co-located sessions; its validation with a non-co-located moderator remains future work. Participants in both studies were volunteers recruited from the authors' university environment, were informed about the

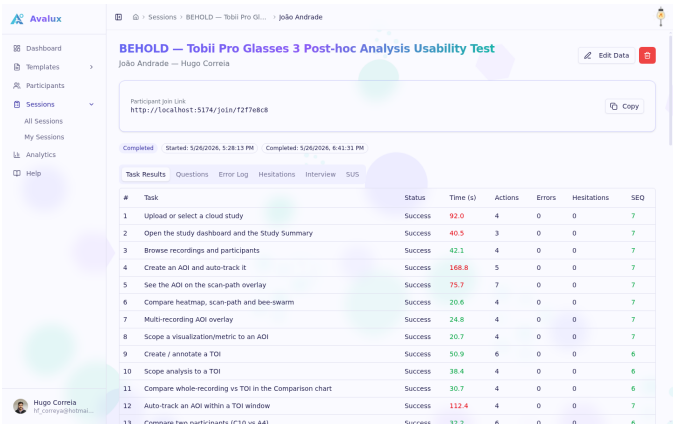

Fig. 4. Session results dashboard: per-task outcomes with time, action, error, hesitation, and SEQ columns; charts below compare performance against the template's optimal baselines.

TABLE I
STUDY 1: SELECTED TASK-LEVEL RESULTS (N=4).

Task	Mean SEQ	Mean time (s)	Note
Wake up panel	1.8	–	all struggled
Recover from error	2.5	–	no error feedback
Set clock to 08:30	–	43.6	10 actions vs. 8 optimal
Read current time	7.0	2.7	100% success

study's purpose and the data collected (including, in study 2, browser-captured screenshots and recordings), and consented to participate; **[complete before submission: ethics approval or exemption statement per the responsible institution, compensation, and consent procedure details.]**

A. Study 1: Household Touch-Panel Controller

Four participants (ages 20–23, two female, three of low and one of high self-rated technical proficiency) attempted 14 tasks, seven simple (e.g. wake the panel, read the time, adjust temperature) and seven complex (e.g. set the clock, change language, recover from an error state), on a commercial water-heater touch panel. Sessions averaged 15 minutes. Aggregate completion across the 4×14 task attempts was approximately 70% success, 20% partial, and 11% failure (percentages independently rounded). SUS scores were 25.0, 40.0, 17.5, and 30.0 (mean 28.1, 95% CI $[13.1, 43.1]$, "Poor" [27]; even the interval's optimistic bound sits far below the benchmark mean of 68 [26]); notably the high-proficiency participant produced the *lowest* score, indicating problems not attributable to inexperience. Task-level metrics located the failures precisely (Table I): the undiscoverable wake-up gesture (SEQ 1.8, all participants struggled), error recovery without any error feedback (SEQ 2.5, participants resorted to power cycling), and clock adjustment (43.6 s and 10 actions against an optimal 8). Participants' reports of buttons changing function across screens figured among the study's principal design recommendations, alongside a persistently visible entry point to settings and explicit error feedback.

TABLE II
STUDY 2: PROTOCOL STRUCTURE (TASK GROUPS AND PER-TASK QUESTION INSTRUMENTS).

Task group	Tasks	Question instruments
Study ingestion	2	choice, open, rating, photo
Data browsing	2	choice, open, rating
Areas of interest (AOI)	4	choice, rating, open, video, photo
Times of interest (TOI)	4	choice, rating, open, video, photo
Visualizations	1	choice, photo
Participant comparison	2	choice, open, rating, photo
Metrics explorer	5	choice, rating, open, audio
Export	1	choice, rating, audio

B. Study 2: Eye-Tracking Analysis Dashboard

The second study evaluated BEHOLD, a post-hoc analysis dashboard for Tobii Pro Glasses 3 recordings, with two participants (ages 22–23, students, low technical proficiency). The protocol exercised the platform's richer features: 21 tasks in eight groups following the analyst's journey (Table II), of which each session ran 20, a 19-item error taxonomy, media question types (screenshots, screen recordings, audio narration), and custom participant fields (handedness, vision correction, device, location).

Task success was near-universal, with one consistent exception: the fixation-metric configuration task in the metrics explorer failed for both participants, isolating the dashboard's least intuitive workflow; every other task was completed successfully by both participants (one task was skipped in one session). Both participants scored the system at SUS 80 ("Good", approaching "Excellent" [27]). Open answers converged on three concrete defects (an auto-track feature hidden in an overflow menu, participant IDs shown instead of names in comparisons, and truncated axis labels), each traceable to a specific task's logs and answers in the report.

C. Cross-Study Reading

Fig. 5 contrasts the per-participant SUS scores of both studies against the industry benchmark. The two cohorts are comparable (young adults in their early twenties, predominantly low technical proficiency), yet the platform separates the two designs by more than fifty SUS points with non-overlapping ranges, and its task-level instruments explain *where* each design earns its score. We take this as evidence that an integrated moderated pipeline can deliver both the discriminative summary metrics and the diagnostic detail that typically require separate tools.

Operationally, a single evaluator ran every session in both studies. The protocol-as-template design meant the second study's considerably richer instrumentation (19 error codes, media questions, custom participant attributes) cost setup effort once rather than per session, and both study reports, including all charts and per-task appendices, were generated without manual assembly. Running real studies also stress-tested the platform itself: study operation exposed a row-level-security gap in the anonymous invitation-counter update (Section III-D) and showed that evaluators under load can

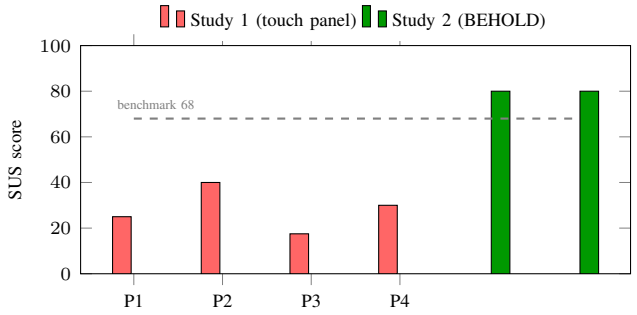

Fig. 5. Per-participant SUS scores in both case studies against the industry benchmark mean of 68 [26]. Participant indices are per study; the two cohorts are unrelated.

leave action counters untracked, both of which fed fixes and roadmap items back into the platform.

V. Discussion and Limitations

Construct validity

Action and error counts are logged by the evaluator rather than instrumented automatically; this keeps the platform applicable to physical devices (study 1 evaluated hardware no browser can instrument) at the cost of observer load and potential undercounting during fast interaction bursts, and single-observer categorization of errors has unmeasured inter-rater reliability. SEQ and SUS are self-report instruments with known ceiling effects. The optimal-path baselines that anchor the efficiency measures are themselves author-defined and inherit the protocol designer's assumptions.

Internal validity

Both studies were run by a single evaluator who is also the platform's author; observer bias in logging and status judgments cannot be excluded. The two case-study cohorts, while demographically similar, attempted different systems with different protocols, so the cross-study SUS contrast (Fig. 5) should be read as discriminative range, not as a controlled comparison.

External validity

The samples (four participants in study 1, two in study 2) are in line with problem-discovery practice [3], [4] but far too small for statistical generalization of the metric values (study 2's two participants in particular make its aggregate figures illustrative only), and both cohorts were young adults in their early twenties. More importantly for a tool paper, we have not yet run a controlled comparison of evaluator effort and data quality against an ad-hoc toolchain or a commercial moderated platform; the case studies demonstrate feasibility and output quality, not superiority. That comparison, in which several evaluators log the same scripted session with Avalux and with a stopwatch-and-spreadsheet baseline, measured for setup time, logging completeness, report latency, and workload, is the designed next study, together with an inter-rater reliability measurement of the live-logging instrument.

Security surface

The anonymous-participant model trades open link joins for RLS policies keyed on unguessable codes; a formal audit of that policy surface (one gap in invitation-counter updates was found and fixed during study operation) is ongoing work.

A further practical consideration is where a hybrid design like this sits between the lab and the fully remote setting. Because the participant client is an ordinary web page, the same protocol runs in a lab (evaluator and participant co-located, as in both case studies), over a video call with the join link shared in chat, or asynchronously through shared links when moderation is not required, although the platform's distinctive instruments (evaluator-logged actions, errors, and hesitations) only apply in the moderated modes. Prior comparisons suggest synchronous remote moderation approaches lab quality [12], [31]; verifying that Avalux's gating protocol preserves that equivalence when the moderator is not physically present is a natural next experiment. Finally, the participant-own-device model inherits the device diversity of field settings [33]: the mobile-first participant client was a direct consequence of pilot participants joining on their phones.

VI. Conclusion and Future Work

Avalux integrates protocol definition, live moderated logging, participant-side questioning on the participant's own device, standardized instruments, analytics, and reporting into one open platform, and carried two complete usability studies whose results discriminated sharply between a poorly and a well-received design.

Future work proceeds on two tracks. The *evaluation* track validates the tool itself: a counterbalanced evaluator study against a stopwatch-and-spreadsheet baseline (setup time, logging completeness, report latency, SUS and workload on the toolchain), an inter-rater reliability study of the live-logging instrument, and, for browser-based systems under test, an optional auto-instrumentation snippet whose event counts can be correlated against evaluator-logged counts to bound manual logging error. The *platform* track continues with a more flexible task model, multi-evaluator observation of a single live session, and localization of the participant client, whose users in both studies preferred answering in their native language; the event timelines, raw data export, order counterbalancing, additional questionnaires, and retroactive anonymization described in Section III were themselves motivated by the two studies and shipped since they concluded.

Availability

The platform is deployed at https://avalux.pt. Its source, comprising the application, the database schema (36 SQL migrations), the seeded protocol of study 2, the in-application tutorial, and the report generator, is available under the MIT license at https://github.com/hugocorreia15/usertesting. Anonymized session data from the case studies is available on request within the limits of the participants' consent.

REFERENCES

[1] J. Brooke, "SUS: A "quick and dirty" usability scale," in *Usability Evaluation in Industry*, P. W. Jordan, B. Thomas, B. A. Weerdmeester, and I. L. McClelland, Eds. London, UK: Taylor & Francis, 1996, pp. 189–194.

[2] J. Sauro and J. S. Dumas, "Comparison of three one-question, post-task usability questionnaires," in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '09)*. Boston, MA, USA: ACM, 2009, pp. 1599–1608.

[3] R. A. Virzi, "Refining the test phase of usability evaluation: How many subjects is enough?" *Human Factors*, vol. 34, no. 4, pp. 457–468, 1992.

[4] J. Nielsen and T. K. Landauer, "A mathematical model of the finding of usability problems," in *Proceedings of the INTERACT '93 and CHI '93 Conference on Human Factors in Computing Systems (INTERCHI '93)*. Amsterdam, The Netherlands: ACM, 1993, pp. 206–213.

[5] UserTesting, Inc., "UserTesting Human Insight Platform," 2026, commercial platform for moderated ("Live Conversation") and unmoderated video-based user research with proprietary participant panel and AI-assisted analysis; enterprise pricing undisclosed. Accessed: 2026-07-13. [Online]. Available: https://www.usertesting.com/platform

[6] ——, "UserZoom UX Research Platform," 2026, former UserZoom quantitative UX research platform, merged into UserTesting in 2023 under Thoma Bravo; userzoom.com permanently redirects to this page. Accessed: 2026-07-13. [Online]. Available: https://www.usertesting.com/platform/userzoom

[7] Maze Design, Inc., "Maze: User Research and Testing Platform," 2026, unmoderated-first prototype and website testing with automated metrics (completion rate, misclick rate, duration, heatmaps), moderated interviews, and AI Moderator; freemium SaaS with enterprise tiers. Accessed: 2026-07-13. [Online]. Available: https://maze.co/

[8] Lookback Group, Inc., "Lookback: Remote User Research Platform," 2026, moderated and unmoderated remote research with screen/camera/audio capture, live stakeholder observation rooms, timestamped notes, and AI-suggested findings; annual per-session plans from USD 299/year. Accessed: 2026-07-13. [Online]. Available: https://www.lookback.com/

[9] Loop11 Pty. Ltd., "Loop11: Online Usability Testing Tool," 2026, task-based moderated and unmoderated testing of live websites and prototypes with screen/webcam/audio recording, heatmaps, clickstreams, and automatic NPS and SUS calculation; subscriptions from ca. USD 199/month. Accessed: 2026-07-13. [Online]. Available: https://www.loop11.com/

[10] Useberry, "Useberry: UX Research & Usability Testing Platform," 2026, prototype-centric unmoderated testing (Figma/ProtoPie) with task completion, time-on-task, drop-off, click-tracking and user-flow analytics, plus recently added moderated interviews; freemium tiered SaaS. Accessed: 2026-07-13. [Online]. Available: https://www.useberry.com/

[11] H. R. Hartson, J. C. Castillo, J. Kelso, and W. C. Neale, "Remote evaluation: The network as an extension of the usability laboratory," in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '96)*. Vancouver, BC, Canada: ACM, 1996, pp. 228–235.

[12] M. S. Andreasen, H. V. Nielsen, S. O. Schrøder, and J. Stage, "What happened to remote usability testing? an empirical study of three methods," in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '07)*. San Jose, CA, USA: ACM, 2007, pp. 1405–1414.

[13] A. Bruun, P. Gull, L. Hofmeister, and J. Stage, "Let your users do the testing: A comparison of three remote asynchronous usability testing methods," in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '09)*. Boston, MA, USA: ACM, 2009, pp. 1619–1628.

[14] O. Alhadreti, "A comparison of synchronous and asynchronous remote usability testing methods," *International Journal of Human–Computer Interaction*, vol. 38, no. 3, pp. 289–297, 2022.

[15] F. Paternò and G. Ballardin, "RemUSINE: A bridge between empirical and model-based evaluation when evaluators and users are distant," *Interacting with Computers*, vol. 13, no. 2, pp. 229–251, 2000.

[16] L. Paganelli and F. Paternò, "Tools for remote usability evaluation of web applications through browser logs and task models," *Behavior Research Methods, Instruments, & Computers*, vol. 35, no. 3, pp. 369–378, 2003.

[17] TechSmith Corporation, "Morae: Usability testing and user experience research software," Software (Recorder, Observer, Manager); discontinued, 2004, https://assets.techsmith.com/docs/pdf-morae/SW-Usability-Testing-with-Morae.pdf.

[18] J. M. C. Bastien, "Usability testing: A review of some methodological and technical aspects of the method," *International Journal of Medical Informatics*, vol. 79, no. 4, pp. e18–e23, 2010.

[19] G. Desolda, G. Gaudino, R. Lanzilotti, S. Federici, and A. Cocco, "UTAssistant: A web platform supporting usability testing in italian public administrations," in *Proceedings of the Doctoral Consortium, Posters and Demos at CHItaly 2017*, ser. CEUR Workshop Proceedings, vol. 1910. CEUR-WS.org, 2017, pp. 138–142.

[20] S. Federici, M. L. Mele, R. Lanzilotti, G. Desolda, M. Bracalenti, F. Meloni, G. Gaudino, A. Cocco, and M. Amendola, "UX evaluation design of UTAssistant: A new usability testing support tool for italian public administrations," in *Human-Computer Interaction. Theories, Methods, and Human Issues (HCII 2018)*, ser. Lecture Notes in Computer Science, vol. 10901. Springer, 2018, pp. 55–67.

[21] A. Generosi, J. Y. Villafan, L. Giraldi, S. Ceccacci, and M. Mengoni, "A test management system to support remote usability assessment of web applications," *Information*, vol. 13, no. 10, p. 505, 2022.

[22] J. H. H. Pedersen, M. Sørensen, J. Stage, and R. T. Høegh, "Introducing asynchronous remote usability testing in practice: An action research project," in *Human-Computer Interaction – INTERACT 2021*, ser. Lecture Notes in Computer Science, vol. 12935. Springer, 2021, pp. 320–338.

[23] International Organization for Standardization, "ISO 9241-11:2018 — ergonomics of human-system interaction — part 11: Usability: Definitions and concepts," International Organization for Standardization, Geneva, Switzerland, Standard, 2018.

[24] A. Bangor, P. T. Kortum, and J. T. Miller, "An empirical evaluation of the system usability scale," *International Journal of Human–Computer Interaction*, vol. 24, no. 6, pp. 574–594, 2008.

[25] J. R. Lewis, "The system usability scale: Past, present, and future," *International Journal of Human–Computer Interaction*, vol. 34, no. 7, pp. 577–590, 2018.

[26] J. Sauro, *A Practical Guide to the System Usability Scale: Background, Benchmarks & Best Practices*. Denver, CO, USA: Measuring Usability LLC, 2011.

[27] A. Bangor, P. Kortum, and J. Miller, "Determining what individual SUS scores mean: Adding an adjective rating scale," *Journal of Usability Studies*, vol. 4, no. 3, pp. 114–123, 2009.

[28] B. Albert and T. Tullis, *Measuring the User Experience: Collecting, Analyzing, and Presenting UX Metrics*, 3rd ed. Cambridge, MA, USA: Morgan Kaufmann, 2022.

[29] J. Sauro and J. R. Lewis, *Quantifying the User Experience: Practical Statistics for User Research*, 2nd ed. Cambridge, MA, USA: Morgan Kaufmann, 2016.

[30] E. McFadden, D. R. Hager, C. J. Elie, and J. M. Blackwell, "Remote usability evaluation: Overview and case studies," *International Journal of Human–Computer Interaction*, vol. 14, no. 3–4, pp. 489–502, 2002.

[31] A. J. B. Brush, M. Ames, and J. Davis, "A comparison of synchronous remote and local usability studies for an expert interface," in *CHI '04 Extended Abstracts on Human Factors in Computing Systems*. Vienna, Austria: ACM, 2004, pp. 1179–1182.

[32] H. B.-L. Duh, G. C. B. Tan, and V. H.-h. Chen, "Usability evaluation for mobile device: A comparison of laboratory and field tests," in *Proceedings of the 8th Conference on Human-Computer Interaction with Mobile Devices and Services (MobileHCI '06)*. Helsinki, Finland: ACM, 2006, pp. 181–186.

[33] A. Kaikkonen, A. Kekäläinen, M. Cankar, T. Kallio, and A. Kankainen, "Usability testing of mobile applications: A comparison between laboratory and field testing," *Journal of Usability Studies*, vol. 1, no. 1, pp. 4–16, 2005.

[34] J. T. Simon-Liedtke, W. K. Bong, T. Schulz, and K. S. Fuglerud, "Remote evaluation in universal design using video conferencing systems during the covid-19 pandemic," in *Universal Access in Human-Computer Interaction. Design Methods and User Experience (HCII 2021)*, ser. Lecture Notes in Computer Science, vol. 12768. Springer, 2021, pp. 116–135.

[35] J. R. Hill, J. C. Brown, N. L. Campbell, and R. J. Holden, "Usability-in-place—remote usability testing methods for homebound older adults: Rapid literature review," *JMIR Formative Research*, vol. 5, no. 11, p. e26181, 2021.

[36] M. Lobchuk, P. R. Bathi, A. Ademeyo, and A. Livingston, "Remote moderator and observer experiences and decision-making during usability testing of a web-based empathy training portal: Content analysis," *JMIR Formative Research*, vol. 6, no. 8, p. e35319, 2022.

[37] S. G. Hart, "NASA-Task Load Index (NASA-TLX); 20 years later," in *Proceedings of the Human Factors and Ergonomics Society Annual Meeting*, vol. 50, no. 9. SAGE, 2006, pp. 904–908.

[38] M. Schrepp, A. Hinderks, and J. Thomaschewski, "Design and evaluation of a short version of the user experience questionnaire (UEQ-S)," *International Journal of Interactive Multimedia and Artificial Intelligence*, vol. 4, no. 6, pp. 103–108, 2017.